

JavaScript Function bind()

[Back to JS page](#)

Table of Contents

- [JavaScript Function bind\(\)](#)
 - [Using bind\(\) to Fix this](#)
 - [Example 1](#)
 - [bind\(\) for Functions Called Later](#)
 - [Example 2](#)
 - [Using bind\(\) for Preserving this](#)
 - [Example 3](#)
 - [bind\(\) with Arguments](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

Using bind() to Fix this

The `bind()` method creates a new function where `this` refers to the object passed as an argument.

With `bind()`, an object can borrow a method from another object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using bind() to Fix this</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

const member = {
  firstName: "Jane",
  lastName: "Smith"
};

// Bind person's method to member object
let fullName = person.fullName.bind(member);
document.getElementById("demo").innerHTML = fullName();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

bind() for Functions Called Later

The `bind()` method is useful when a function is called later (like in a callback) and you need `this` to refer to the correct object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>bind() for Functions Called Later</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  display: function() {
    document.getElementById("demo").innerHTML = this.firstName + " " + this.lastName;
  }
};

// Without bind - setTimeout would lose "this" context
// setTimeout(person.display, 3000); // this would be window

// With bind - "this" is properly preserved
setTimeout(person.display.bind(person), 3000);
document.getElementById("demo").innerHTML = "Waiting 3 seconds...";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using bind() for Preserving this

When you pass an object method as a callback, `this` is lost. `bind()` preserves it:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using bind() for Preserving this</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  display: function() {
    document.getElementById("demo").innerHTML = this.firstName + " " + this.lastName;
  }
};

// Without bind, "this" in the callback would refer to the button element
// document.getElementById("myBtn").addEventListener("click", person.display); // Would show "un

// With bind, "this" correctly refers to person
document.getElementById("myBtn").addEventListener("click", person.display.bind(person));
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

bind() with Arguments

`bind()` can also accept arguments, creating a new function with preset parameters (partial application):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function bind()</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>bind() with Arguments</h4>
<p id="demo"></p>

<script>
function multiply(a, b) {
    return a * b;
}

// Create a new function by binding the first argument
let multiplyBy2 = multiply.bind(this, 2);
let multiplyBy10 = multiply.bind(this, 10);

document.getElementById("demo").innerHTML =
    "multiplyBy2(5): " + multiplyBy2(5) + "<br>" +
    "multiplyBy10(5): " + multiplyBy10(5);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Function bind\(\)](#).